

Pre-trained 모델 활용

05 대규모 언어 모델

목차

—
언어 모델의 개념을
알아보고 가장 대표적인
활용 방법인 대화형 텍스트
생성 모델을 사용하는
방법을 알아봅니다.

01 대규모 언어 모델

02 모델 사용하기

01

대규모 언어 모델

④ 언어 모델(Language Model)

단어의 배열(문장)마다 해당 문장이 적절할 확률을 할당하는 모델

- 각 문장마다 확률을 할당하여 적절한 문장을 찾아내는 모델
- 기계 번역, 오타 교정, 음성 인식, 빈칸 완성 등의 작업을 수행할 수 있음

④ 문장과 확률

"I'm a ? ."

"I'm a boy ." 일 확률

V

"I'm a bus ." 일 확률

- 학습을 통해 자연스러운 문장일 확률을 예측
- 가장 적절할 확률이 높은 문장을 선택
- 버스보다는 소년이 자연스럽기 때문에 전자에 더 높은 확률이 할당됨

④ 가중치와 모델 성능

많은 가중치를 가진 모델일수록 복잡하고 많은 내용을 학습할 수 있음

- 모델의 가중치가 많을수록 모델의 기억력이 높아짐.
- 복잡한 문제를 해결하려면 일반적으로 더 많은 가중치가 필요함

④ 언어모델의 성능

다양한 문장의 확률을 인간과 비슷하게 할당할 수 있는 정도

- 많은 문장의 확률을 정확하게 예측할수록 인간과 비슷한 문장을 생성할 수 있는 모델
- 많은 문장을 학습하려면 많은 가중치가 필요함

④ 대규모 언어 모델(Large Language Model)

모델의 크기와 학습 데이터의 양을 매우 크게 확장한 언어 모델

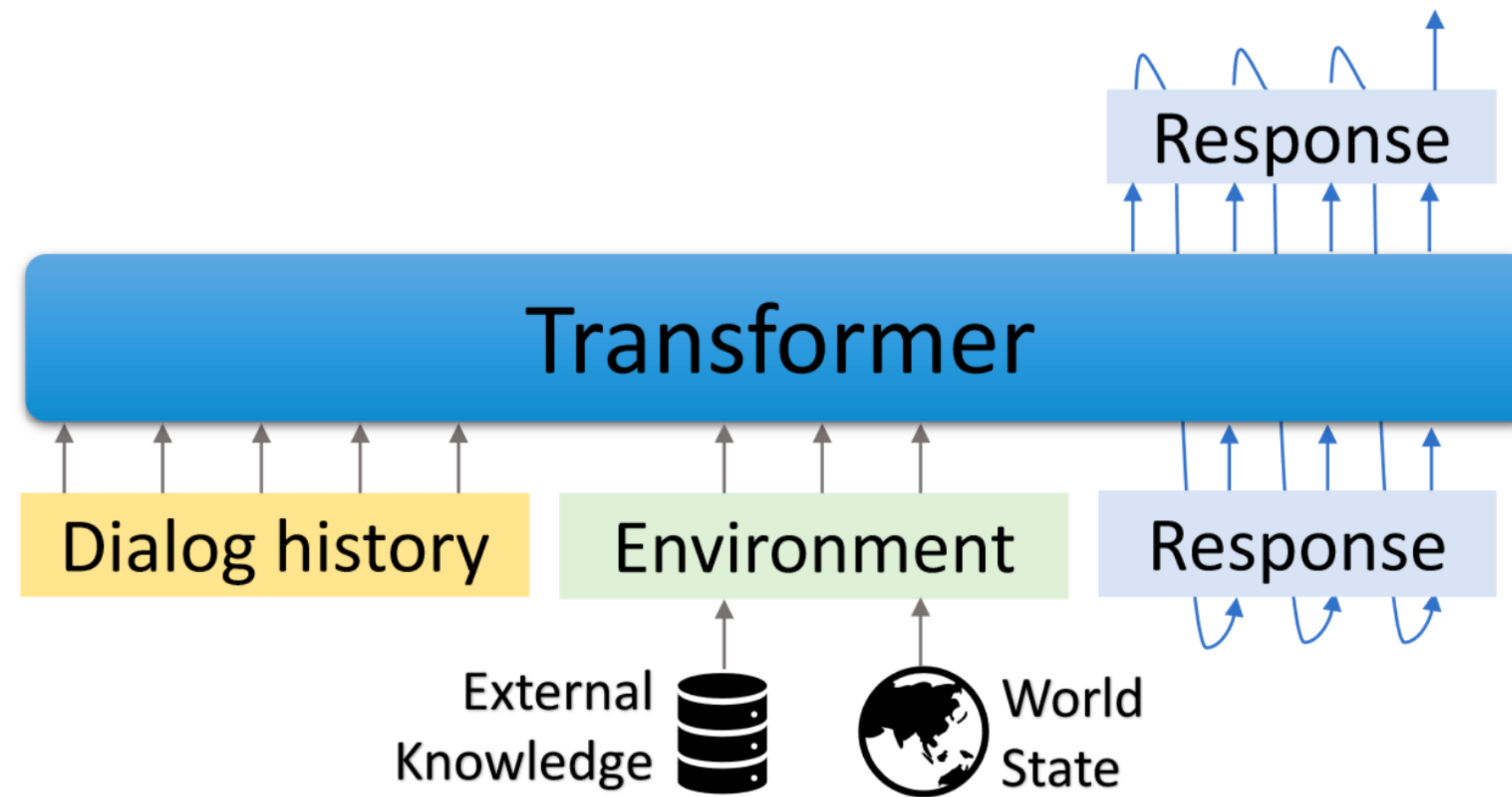
- 언어 모델의 크기를 극적으로 크게 만들고 아주 다양한 종류의 문서와 웹 페이지를 학습시킴
- 방대한 분야에 대해 문장이 적절할 확률을 예측할 수 있는 모델

☑ GODEL (Grounded Open Dialogue Language Model)

사전 지식에 기반한 대화를 위한 언어 모델

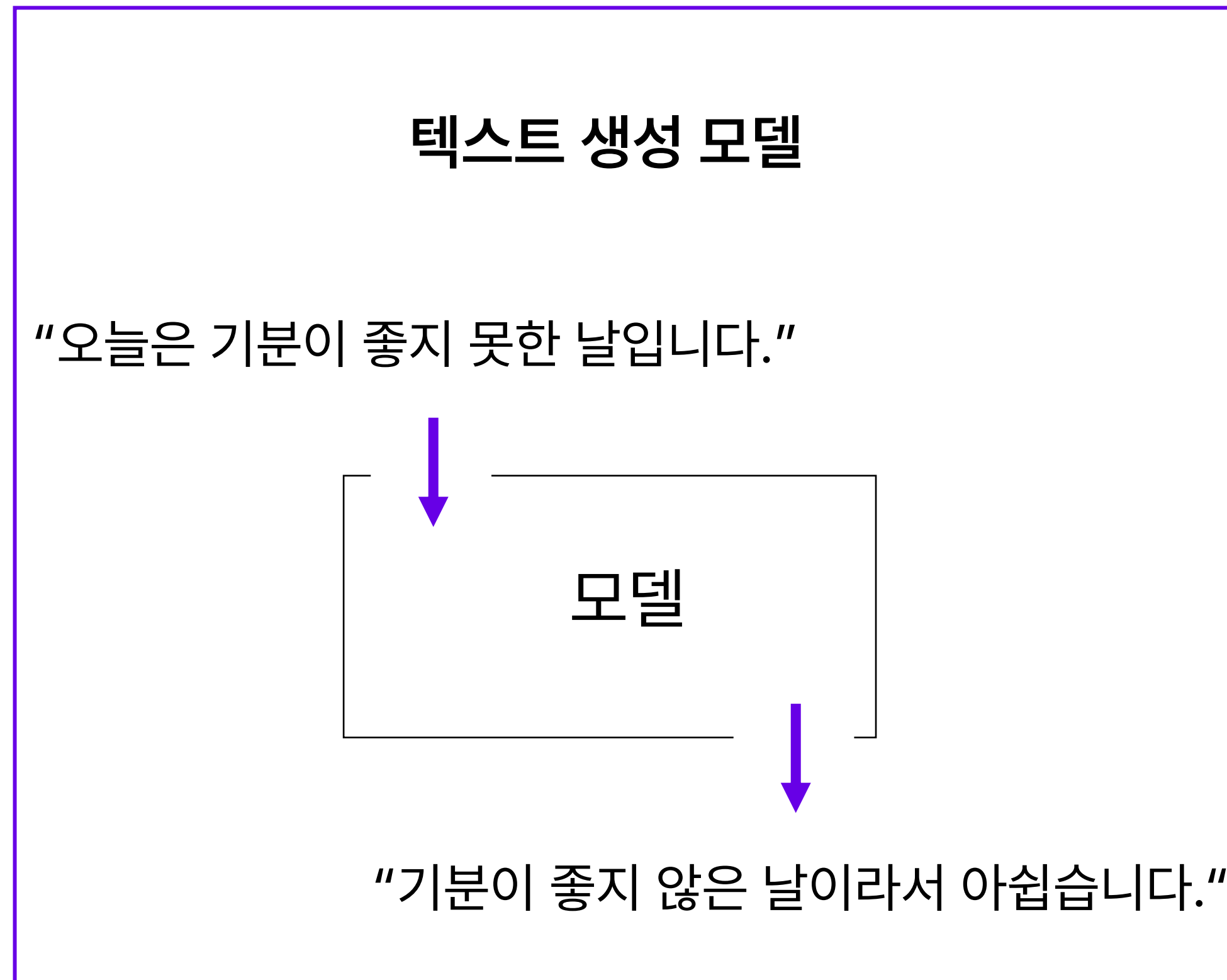
- 지식을 기반으로 대화를 할 수 있는 언어 모델로 Microsoft Research에서 2022년에 개발한 모델
- 외부의 지식을 참조하여 대답할 수 있으며 원하는 목표로 조정하기 쉬움
- 학습되지 않은 새로운 지식도 대화 중에 첨가하여 문장을 완성할 수 있음

☑ GODEL의 구조



- 지금까지의 대화 기록, 외부 지식을 기반으로 답변을 생성하는 모델

④ 텍스트 생성 모델



④ 텍스트 생성 모델

"오늘은 기분이 좋지 못한 날입니다."

인코딩

(숫자들의 행렬)



모델

(숫자들의 행렬)

디코딩

"기분이 좋지 않은 날이라서 아쉽습니다."

- 입력과 출력이 모두 행렬인 모델
- 자연어를 숫자 행렬로 바꾸는 과정: 인코딩
- 숫자 행렬을 자연어로 바꾸는 과정: 디코딩

02

모델 사용하기

④ 인코딩과 디코딩

“오늘은 기분이 좋지 못한 날입니다.”

↓ 인코딩

(숫자들의 행렬)



(숫자들의 행렬)

↓ 디코딩

“기분이 좋지 않은 날이라서 아쉽습니다.”

④ 허깅페이스 토크나이저 클래스(Tokenizer Class)

인코딩과 디코딩을 처리하는 클래스

- 자연어 데이터를 입력 받으면 토큰화, 벡터화를 자동으로 진행
- 일반적으로 모델마다 사용한 토크나이저가 다름
- 토크나이저를 바꾸면 다시 학습을 진행해야 함

👉 토크나이저 불러오기

```
from transformers import AutoTokenizer  
  
tokenizer = AutoTokenizer.from_pretrained("microsoft/GODEL-v1_1-large-seq2seq")
```

- 모델을 불러올 때와 유사하게 from_pretrained 함수를 사용
- 토크나이저도 데이터 셋에 따라 학습이 필요하므로 사전 학습된 가중치를 가져오거나 학습을 해야 함

02 모델 사용하기

④ 토큰으로 나누기

```
raw_text = "I am a boy"
tokens = tokenizer.tokenize(raw_text)
print(tokens)          # ['_I', '_am', '_', 'a', '_boy']
```

- 단어를 정해진 토큰 단위로 나누는 함수
- 나뉘어진 토큰들의 행렬을 반환하는 함수

02 모델 사용하기

④ 숫자로 바꾸기

```
raw_text = "I am a boy"
ids = tokenizer.convert_tokens_to_ids(tokens)
print(ids)                # [27, 183, 3, 9, 4940]
```

- 토큰을 정해진 규칙에 따라 숫자들의 행렬로 변환하는 과정
- 이 두가지 과정을 인코딩(Encoding)이라고 함

④ 인코딩 (Encoding)

```
raw_text = "I am a boy"
tokenizer(raw_text)
# {'input_ids': [27, 183, 3, 9, 4940, 1], 'attention_mask': [1, 1, 1, 1, 1, 1]}
```

- 토큰라이저를 직접 호출하면서 자연어 데이터를 전달하면 인코딩을 할 수 있음
- input_ids가 모델에 입력해야 하는 값

④ 디코딩 (Decoding)

```
encoded = tokenizer(raw_text, return_tensors="pt")
input_ids = encoded.input_ids
print(input_ids) # tensor([[ 27,  183,    3,    9, 4940,    1]])

dec_text = tokenizer.decode(input_ids[0])
print(dec_text) # I am a boy</s>
```

- 숫자들의 배열을 다시 자연어로 복구하는 과정
- decode 함수를 이용
- 모델의 출력 값에도 적용해야 함

☑ 모델 불러오기

```
model = AutoModelForSeq2SeqLM.from_pretrained("microsoft/GODEL-v1_1-large-seq2seq").to("cuda")
```

- 동일하게 from_pretrained를 사용
- to()함수는 pytorch에서 객체가 동작할 위치를 정하는 함수
- to("cpu"): CPU에서 동작
- to("cuda"): GPU에서 동작

02 모델 사용하기

☑ GPU로 보내기

```
model = AutoModelForSeq2SeqLM.from_pretrained("microsoft/GODEL-v1_1-large-seq2seq").to("cuda") #gpu로 이동  
input_ids = input_ids.to("cuda") # gpu로 이동
```

- 연산을 하려면 모델과 데이터를 포함한 모든 것이 같은 장치에 있어야 함
- 입력 데이터도 GPU로 이동

02 모델 사용하기

④ 추론하기

```
output = model.generate(input_ids, max_length=128) # 추론
print(output)
# tensor([[ 0, 27, 183, 3, 9, 3202, 11, 27, 183, 3, 9, 4940, 1]], device='cuda:0')

dec_text = tokenizer.decode(output[0]) # 디코딩
print(dec_text) # <pad> I am a girl and I am a boy</s>
```

- generate 함수를 이용해 추론을 진행
- 결과를 자연어로 바꾸려면 디코딩이 필요함

☑ GODEL의 학습 데이터

```
{  
  "Context": "Please remind me of calling to Jessie at 2PM.",  
  "Knowledge": "reminder_contact_name is Jessie, reminder_time is 2PM",  
  "Response": "Sure, set the reminder: call to Jesse at 2PM"  
}
```

- Context: 하던 대화 내용
- Knowledge: 사전 지식
- Response: 원하는 응답

④ 권장 입력 형태

```
Instruction: given a dialog context and related knowledge, you need to response safely based on the knowledge.  
[CONTEXT] How much is the price for a pizza?  
[KNOWLEDGE] pizza is 8 dollars.
```

- Instruction: 지시사항
- [CONTEXT] 는 특수한 토큰으로 뒤에는 하던 대화 내용
- [KNOWLEDGE] 는 특수한 토큰으로 뒤에는 사전 지식으로 전달할 내용